

Компьютерная графика

Лекция 3: Буферы на GPU, атрибуты вершин,
индексированный рендеринг, instancing

2026

- Мы знаем, что *вершинный шейдер* обрабатывает по одной *вершины* нашей 3D (или 2D) модели

- Мы знаем, что *вершинный шейдер* обрабатывает по одной *вершины* нашей 3D (или 2D) модели
- Пока мы умеем только хардкодить их *атрибуты* (положение в пространстве, цвет) в шейдере

- Мы знаем, что *вершинный шейдер* обрабатывает по одной *вершины* нашей 3D (или 2D) модели
- Пока мы умеем только хардкодить их *атрибуты* (положение в пространстве, цвет) в шейдере
- В модели могут быть тысячи или миллионы вершин, одним шейдером хочется рисовать разные модели

- Мы знаем, что *вершинный шейдер* обрабатывает по одной *вершины* нашей 3D (или 2D) модели
- Пока мы умеем только хардкодить их *атрибуты* (положение в пространстве, цвет) в шейдере
- В модели могут быть тысячи или миллионы вершин, одним шейдером хочется рисовать разные модели
- $\implies$ Хочется как-то загрузить их в память GPU

- Мы знаем, что *вершинный шейдер* обрабатывает по одной *вершины* нашей 3D (или 2D) модели
- Пока мы умеем только хардкодить их *атрибуты* (положение в пространстве, цвет) в шейдере
- В модели могут быть тысячи или миллионы вершин, одним шейдером хочется рисовать разные модели
- $\implies$ Хочется как-то загрузить их в память GPU
- $\implies$ Хочется как-то рассказать шейдеру, где они лежат

- Могут хранить произвольные данные на GPU, сами никак их не интерпретируют

- Могут хранить произвольные данные на GPU, сами никак их не интерпретируют
- При создании нужно указать:

- Могут хранить произвольные данные на GPU, сами никак их не интерпретируют
- При создании нужно указать:
 - `size` – размер в байтах

- Могут хранить произвольные данные на GPU, сами никак их не интерпретируют
- При создании нужно указать:
 - `size` – размер в байтах
 - `usage` – флаги, описывающие, как мы будем использовать этот буфер (нужны для валидации, выбора правильного вида памяти, и т.д.)

- Возможные значения `usage` (это битовые флаги – их можно комбинировать):

- Возможные значения `usage` (это битовые флаги – их можно комбинировать):
 - `MapRead`, `MapWrite` – можно записать на чтение/запись (об этом позже)

- Возможные значения `usage` (это битовые флаги – их можно комбинировать):
 - `MapRead`, `MapWrite` – можно записать на чтение/запись (об этом позже)
 - `CopySrc`, `CopyDst` – можно копировать данные из/в этот буффер

- Возможные значения `usage` (это битовые флаги – их можно комбинировать):
 - `MapRead`, `MapWrite` – можно записать на чтение/запись (об этом позже)
 - `CopySrc`, `CopyDst` – можно копировать данные из/в этот буффер
 - `Index`, `Vertex` – можно использовать для индексов / самих вершин

- Возможные значения `usage` (это битовые флаги – их можно комбинировать):
 - `MapRead`, `MapWrite` – можно записать на чтение/запись (об этом позже)
 - `CopySrc`, `CopyDst` – можно копировать данные из/в этот буффер
 - `Index`, `Vertex` – можно использовать для индексов / самих вершин
 - `Uniform` – можно использовать для *uniform значений* (как `immediates`, но больше объём)

- Возможные значения `usage` (это битовые флаги – их можно комбинировать):
 - `MapRead`, `MapWrite` – можно записать на чтение/запись (об этом позже)
 - `CopySrc`, `CopyDst` – можно копировать данные из/в этот буффер
 - `Index`, `Vertex` – можно использовать для индексов / самих вершин
 - `Uniform` – можно использовать для *uniform значений* (как *immediates*, но больше объём)
 - `Storage` – можно использовать для *random access storage*

Buffer usage

- Возможные значения `usage` (это битовые флаги – их можно комбинировать):
 - `MapRead`, `MapWrite` – можно записать на чтение/запись (об этом позже)
 - `CopySrc`, `CopyDst` – можно копировать данные из/в этот буффер
 - `Index`, `Vertex` – можно использовать для индексов / самих вершин
 - `Uniform` – можно использовать для *uniform значений* (как *immediates*, но больше объём)
 - `Storage` – можно использовать для *random access storage*
 - `Indirect` – можно использовать для *indirect draw* команд

Buffer usage

- Возможные значения `usage` (это битовые флаги – их можно комбинировать):
 - `MapRead`, `MapWrite` – можно записать на чтение/запись (об этом позже)
 - `CopySrc`, `CopyDst` – можно копировать данные из/в этот буффер
 - `Index`, `Vertex` – можно использовать для индексов / самих вершин
 - `Uniform` – можно использовать для *uniform значений* (как *immediates*, но больше объём)
 - `Storage` – можно использовать для *random access storage*
 - `Indirect` – можно использовать для *indirect draw* команд
 - `QueryResolve` – можно использовать для сохранения результатов *occlusion/timestamp queries*

- Создание буфера работает примерно как `malloc`: память выделяется один раз, перевыделить её нельзя

- Создание буфера работает примерно как `malloc`: память выделяется один раз, перевыделить её нельзя
- Если вам нужен буфер другого размера, придётся его пересоздать

- Создание буфера работает примерно как `malloc`: память выделяется один раз, перевыделить её нельзя
- Если вам нужен буфер другого размера, придётся его пересоздать
- Для динамических данных (создаются/удаляются объекты и т.п.) придётся руками реализовывать обновление размера (примерно как в `std::vector`)

Буферы: запись данных

- Записать данные в буфер можно через queue:

```
void wgpuQueueWriteBuffer(WGPUQueue queue, WGPUBuffer buffer,  
    uint64_t bufferOffset, void const * data, size_t size);
```

- `buffer` – в какой буфер грузим данные
- `bufferOffset` – начиная с какого байта в буфере загрузить данные
- `data` – указатель на данные
- `size` – размер данных в байтах

- Так же, как с другими операциями, `queue.writeBuffer` не пишет сразу в память GPU, а кладёт в очередь команду записи

- Так же, как с другими операциями, `queue.writeBuffer` не пишет сразу в память GPU, а кладёт в очередь команду записи
- После вызова `queue.writeBuffer` с данными по указателю `data` можно делать всё, что угодно (в т.ч. удалить)

- Так же, как с другими операциями, `queue.writeBuffer` не пишет сразу в память GPU, а кладёт в очередь команду записи
- После вызова `queue.writeBuffer` с данными по указателю `data` можно делать всё, что угодно (в т.ч. удалить)
- $\implies$ WebGPU сначала копирует данные в собственную память, а потом инициирует отложенную загрузку на GPU

- Если мы обновляем данные в буфере каждый кадр, GPU придётся ждать, пока буфер перестанет быть нужен, чтобы обновить в нём данные

- Если мы обновляем данные в буфере каждый кадр, GPU придётся ждать, пока буфер перестанет быть нужен, чтобы обновить в нём данные
- $\implies$ WebGPU, скорее всего, внутри будет делать двойную (или тройную) буферизацию – иметь несколько копий буфера, циклически переиспользующихся от кадра к кадру

Mapped buffer

- Можно получить виртуальный указатель на буфер или его часть и пользоваться им для чтения/записи:

```
buffer.mapAsync(mode, offset, size)
```

Mapped buffer

- Можно получить виртуальный указатель на буфер или его часть и пользоваться им для чтения/записи:

`buffer.mapAsync(mode, offset, size)`

- `mode` – комбинация флагов `READ` и `WRITE`

Mapped buffer

- Можно получить виртуальный указатель на буфер или его часть и пользоваться им для чтения/записи:

`buffer.mapAsync(mode, offset, size)`

- `mode` – комбинация флагов `READ` и `WRITE`
- `offset, size` – какой диапазон буфера хотим замапить

Mapped buffer

- Можно получить виртуальный указатель на буфер или его часть и пользоваться им для чтения/записи:

`buffer.mapAsync(mode, offset, size)`

- `mode` – комбинация флагов `READ` и `WRITE`
 - `offset, size` – какой диапазон буфера хотим замапить
- Эта операция *асинхронная*: принимает *callback*, который будет вызван, когда результат будет готов

Mapped buffer

- Можно получить виртуальный указатель на буфер или его часть и пользоваться им для чтения/записи:

`buffer.mapAsync(mode, offset, size)`

- `mode` – комбинация флагов `READ` и `WRITE`
- `offset, size` – какой диапазон буфера хотим замапить
- Эта операция *асинхронная*: принимает *callback*, который будет вызван, когда результат будет готов
- Когда буфер запамлен:
 - `buffer.getMappedRange/get_mapped_range_mut` – получить указатель на чтение/запись
 - `buffer.getConstMappedRange/get_mapped_range` – получить указатель на чтение

Mapped buffer

- Можно получить виртуальный указатель на буфер или его часть и пользоваться им для чтения/записи:
`buffer.mapAsync(mode, offset, size)`
 - `mode` – комбинация флагов `READ` и `WRITE`
 - `offset, size` – какой диапазон буфера хотим замапить
- Эта операция *асинхронная*: принимает *callback*, который будет вызван, когда результат будет готов
- Когда буфер запамлен:
 - `buffer.getMappedRange/get_mapped_range_mut` – получить указатель на чтение/запись
 - `buffer.getConstMappedRange/get_mapped_range` – получить указатель на чтение
- После использования буфер нужно отмапить: `buffer.unmap()`

Mapped buffer

- Можно получить виртуальный указатель на буфер или его часть и пользоваться им для чтения/записи:
`buffer.mapAsync(mode, offset, size)`
 - `mode` – комбинация флагов `READ` и `WRITE`
 - `offset, size` – какой диапазон буфера хотим замапить
- Эта операция *асинхронная*: принимает *callback*, который будет вызван, когда результат будет готов
- Когда буфер запамлен:
 - `buffer.getMappedRange/get_mapped_range_mut` – получить указатель на чтение/запись
 - `buffer.getConstMappedRange/get_mapped_range` – получить указатель на чтение
- После использования буфер нужно отмапить: `buffer.unmap()`
- Пока буфер замаплен, работать с буфером (загружать данные другими методами, использовать данные для рисования) нельзя

- Mapped buffers работают через UMA/ReBAR, где это поддерживается, либо через явное копирование CPU ↔ GPU

Mapped buffer

- Mapped buffers работают через UMA/ReBAR, где это поддерживается, либо через явное копирование CPU ↔ GPU
- Единственный способ *прочитать* что-то из GPU буфера на CPU

Mapped buffer

- Mapped buffers работают через UMA/ReBAR, где это поддерживается, либо через явное копирование CPU ↔ GPU
- Единственный способ *прочитать* что-то из GPU буфера на CPU
- Буфер можно создать с флагом `mappedAtCreation`, тогда он будет сразу замаплен (без асинхронного ожидания), – удобно для буферов, данные которых загружаются только один раз при создании

Буферы: типичный пример создания

```
// Псевдокод :)

vertex_buffer = device.create_buffer(
    size = vertices.size() * sizeof(vertex),
    usage = VERTEX | COPY_DST
);

queue.write_buffer(8vertex_buffer, 0, vertex_buffer.size());
```

- Вершины – основные входные данные графического конвейера

- Вершины – основные входные данные графического конвейера
- С точки зрения графических API, вершины характеризуются набором *атрибутов*: позиция, цвет, нормаль, текстурные координаты, и т.п.

- Вершины – основные входные данные графического конвейера
- С точки зрения графических API, вершины характеризуются набором *атрибутов*: позиция, цвет, нормаль, текстурные координаты, и т.п.
- Каждый атрибут описывается отдельно, может иметь свой тип и формат хранения, – всё это запоминается в *render pipeline*

Атрибуты вершин: шейдер

```
struct VertexIn
{
    @location(0) position : vec3f,
    @location(1) normal : vec3f,
    @location(2) color : vec4f,
    @location(3) texcoord : vec2f,
}
```

Атрибуты вершин: шейдер

```
struct VertexIn
{
    @location(0) position : vec3f,
    @location(1) normal : vec3f,
    @location(2) color : vec4f,
    @location(3) texcoord : vec2f,
}
```

- Location: 0, 1, ... maxVertexAttributes - 1

Атрибуты вершин: шейдер

```
struct VertexIn
{
    @location(0) position : vec3f,
    @location(1) normal : vec3f,
    @location(2) color : vec4f,
    @location(3) texcoord : vec2f,
}
```

- Location: 0, 1, ... maxVertexAttributes - 1
- maxVertexAttributes $\geq$ 16

Атрибуты вершин: шейдер

```
struct VertexIn
{
    @location(0) position : vec3f,
    @location(1) normal : vec3f,
    @location(2) color : vec4f,
    @location(3) texcoord : vec2f,
}
```

- Location: 0, 1, ... maxVertexAttributes - 1
- maxVertexAttributes $\geq$ 16
- Есть тип (f32/i32/u32) и размерность (1/2/3/4)

Floating-point (f32/vec2f/...) форматы:

	16-bit	32-bit
1D	Float16	Float32
2D	Float16x2	Float32x2
3D		Float32x2
4D	Float16x4	Float32x4

Unsigned integer (u32/vec2u/...) форматы:

	8-bit	16-bit	32-bit
1D	Uint8	Uint16	Uint32
2D	Uint8x2	Uint16x2	Uint32x2
3D			Uint32x3
4D	Uint8x4	Uint16x4	Uint32x4

Signed integer (**i32**/vec2i/...) форматы:

	8-bit	16-bit	32-bit
1D	Sint8	Sint16	Sint32
2D	Sint8x2	Sint16x2	Sint32x2
3D			Sint32x3
4D	Sint8x4	Sint16x4	Sint32x4

Атрибуты вершин: нормированные форматы

- Floating-point форматы часто слишком расточительны: большой диапазон, относительная точность

Атрибуты вершин: нормированные форматы

- Floating-point форматы часто слишком расточительны: большой диапазон, относительная точность
- Например, цвет часто хранится 8-битным числом 0..255 и интерпретируется как число $[0, 1]$ (по формуле $x / 255.0$)

Атрибуты вершин: нормированные форматы

- Floating-point форматы часто слишком расточительны: большой диапазон, относительная точность
- Например, цвет часто хранится 8-битным числом 0..255 и интерпретируется как число $[0, 1]$ (по формуле $x / 255.0$)
- Такие форматы называются *нормированными*: хранятся как знаковое/беззнаковое целое, но в шейдер попадают как floating-point (преобразование за нас делает GPU)

Атрибуты вершин: нормированные форматы

- Floating-point форматы часто слишком расточительны: большой диапазон, относительная точность
- Например, цвет часто хранится 8-битным числом 0..255 и интерпретируется как число $[0, 1]$ (по формуле $x / 255.0$)
- Такие форматы называются *нормированными*: хранятся как знаковое/беззнаковое целое, но в шейдер попадают как floating-point (преобразование за нас делает GPU)
 - Для беззнаковых: $[0, 2^{bits} - 1] \rightarrow [0, 1]$ по формуле $x / \text{pow}(2, bits)$

Атрибуты вершин: нормированные форматы

- Floating-point форматы часто слишком расточительны: большой диапазон, относительная точность
- Например, цвет часто хранится 8-битным числом 0..255 и интерпретируется как число $[0, 1]$ (по формуле $x / 255.0$)
- Такие форматы называются *нормированными*: хранятся как знаковое/беззнаковое целое, но в шейдер попадают как floating-point (преобразование за нас делает GPU)
 - Для беззнаковых: $[0, 2^{bits} - 1] \rightarrow [0, 1]$ по формуле $x / \text{pow}(2, bits)$
 - Для знаковых: $[-2^{bits-1}, 2^{bits-1} - 1] \rightarrow [-1, 1]$ по формуле $\text{clamp}(x / (\text{pow}(2, bits-1) - 1), -1, 1)$

Атрибуты вершин: нормированные форматы

- Floating-point форматы часто слишком расточительны: большой диапазон, относительная точность
- Например, цвет часто хранится 8-битным числом 0..255 и интерпретируется как число $[0, 1]$ (по формуле $x / 255.0$)
- Такие форматы называются *нормированными*: хранятся как знаковое/беззнаковое целое, но в шейдер попадают как floating-point (преобразование за нас делает GPU)
 - Для беззнаковых: $[0, 2^{bits} - 1] \rightarrow [0, 1]$ по формуле $x / \text{pow}(2, bits)$
 - Для знаковых: $[-2^{bits-1}, 2^{bits-1} - 1] \rightarrow [-1, 1]$ по формуле $\text{clamp}(x / (\text{pow}(2, bits-1) - 1), -1, 1)$
- Полезно для компактного хранения атрибутов (цвет, нормали)

Атрибуты вершин: нормированные форматы

- Floating-point форматы часто слишком расточительны: большой диапазон, относительная точность
- Например, цвет часто хранится 8-битным числом 0..255 и интерпретируется как число $[0, 1]$ (по формуле $x / 255.0$)
- Такие форматы называются *нормированными*: хранятся как знаковое/беззнаковое целое, но в шейдер попадают как floating-point (преобразование за нас делает GPU)
 - Для беззнаковых: $[0, 2^{bits} - 1] \rightarrow [0, 1]$ по формуле $x / \text{pow}(2, bits)$
 - Для знаковых: $[-2^{bits-1}, 2^{bits-1} - 1] \rightarrow [-1, 1]$ по формуле $\text{clamp}(x / (\text{pow}(2, bits-1) - 1), -1, 1)$
- Полезно для компактного хранения атрибутов (цвет, нормали)
- Имеет смысл *только* для floating-point атрибутов

Атрибуты вершин: нормированные форматы

Unsigned integer нормированные форматы:

	8-bit	16-bit	32-bit
1D	Unorm8	Unorm16	Unorm32
2D	Unorm8x2	Unorm16x2	Unorm32x2
3D			Unorm32x3
4D	Unorm8x4	Unorm16x4	Unorm32x4

Атрибуты вершин: нормированные форматы

Signed integer нормированные форматы:

	8-bit	16-bit	32-bit
1D	Snorm8	Snorm16	Snorm32
2D	Snorm8x2	Snorm16x2	Snorm32x2
3D			Snorm32x3
4D	Snorm8x4	Snorm16x4	Snorm32x4

- Разные атрибуты могут лежать в разных буферах, поэтому в `renderPipelineDescriptor.vertex` описывается массив *buffer layouts*

Атрибуты вершин: буферы

- Разные атрибуты могут лежать в разных буферах, поэтому в `renderPipelineDescriptor.vertex` описывается массив *buffer layouts*
- Каждый *buffer layout* описывает массив атрибутов, лежащих вместе в одном буфере

Атрибуты вершин: буферы

- Разные атрибуты могут лежать в разных буферах, поэтому в `renderPipelineDescriptor.vertex` описывается массив *buffer layouts*
- Каждый *buffer layout* описывает массив атрибутов, лежащих вместе в одном буфере
- У каждого атрибута есть сдвиг (*offset*) от начала буфера до значения атрибута нулевой вершины

Атрибуты вершин: буферы

- Разные атрибуты могут лежать в разных буферах, поэтому в `renderPipelineDescriptor.vertex` описывается массив *buffer layouts*
- Каждый *buffer layout* описывает массив атрибутов, лежащих вместе в одном буфере
- У каждого атрибута есть сдвиг (*offset*) от начала буфера до значения атрибута нулевой вершины
- У всех атрибутов одного буфера есть общее расстояние (*stride*) между значениями атрибутов соседних вершин

Атрибуты вершин: stride

- Атрибут вершины с номером i будет взят по смещению $\text{offset} + \text{stride} * i$

Атрибуты вершин: stride

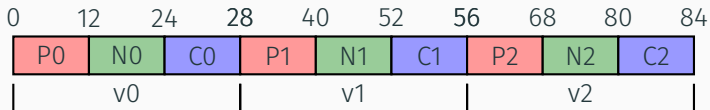
- Атрибут вершины с номером i будет взят по смещению $\text{offset} + \text{stride} * i$
 - Нулевая вершина: offset
 - Первая вершина: $\text{offset} + \text{stride}$
 - Вторая вершина: $\text{offset} + 2 * \text{stride}$
 - И т.д.

Атрибуты вершин: stride

- Атрибут вершины с номером i будет взят по смещению $\text{offset} + \text{stride} * i$
 - Нулевая вершина: offset
 - Первая вершина: $\text{offset} + \text{stride}$
 - Вторая вершина: $\text{offset} + 2 * \text{stride}$
 - И т.д.
- Нужен для гибкости хранения атрибутов

Атрибуты вершин: пример array-of-structs, один буфер

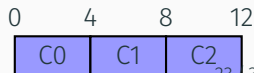
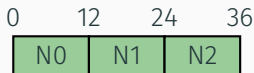
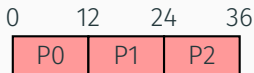
```
struct vertex {  
    float position[3];  
    float normal[3];  
    unsigned char color[4];  
};  
  
vertex_buffers = [  
    {  
        stride: sizeof(vertex),  
        attributes: [  
            { format: Float32x3, offset: 0, location: 0 },  
            { format: Float32x3, offset: 12, location: 1 },  
            { format: Unorm8x4, offset: 24, location: 2 },  
        ],  
    },  
];
```



Атрибуты вершин: пример struct-of-arrays, три буфера

```
float positions[3 * N];
float normals[3 * N];
unsigned char colors[4 * N];

vertex_buffers = [
    {
        stride: 12,
        attributes: [ { format: Float32x3, offset: 0, location: 0 } ],
    },
    {
        stride: 12,
        attributes: [ { format: Float32x3, offset: 0, location: 1 } ],
    },
    {
        stride: 4,
        attributes: [ { format: Unorm8x4, offset: 0, location: 2 } ],
    },
];
```



Аттрибуты вершин: как хранить?

- По смыслу, каждый *buffer layout* описывает массив структур (*stride* – размер структуры)

Аттрибуты вершин: как хранить?

- По смыслу, каждый *buffer layout* описывает массив структур (*stride* – размер структуры)
- По умолчанию используйте один буфер со структурой вершины (например, если вершины хранятся непрерывно в памяти, e.g. `std::vector<vertex>`)

Аттрибуты вершин: как хранить?

- По смыслу, каждый *buffer layout* описывает массив структур (*stride* – размер структуры)
- По умолчанию используйте один буфер со структурой вершины (например, если вершины хранятся непрерывно в памяти, e.g. `std::vector<vertex>`)
- Если часть атрибутов постоянны, а другую часть нужно иногда обновлять – разделите эти части по двум разным буферам

Аттрибуты вершин: как хранить?

- По смыслу, каждый *buffer layout* описывает массив структур (*stride* – размер структуры)
- По умолчанию используйте один буфер со структурой вершины (например, если вершины хранятся непрерывно в памяти, e.g. `std::vector<vertex>`)
- Если часть атрибутов постоянны, а другую часть нужно иногда обновлять – разделите эти части по двум разным буферам
- Если часть атрибутов не нужны для некоторых алгоритмов (например, для теней) – выделите их в отдельный буфер

Аттрибуты вершин: как хранить?

- По смыслу, каждый *buffer layout* описывает массив структур (*stride* – размер структуры)
- По умолчанию используйте один буфер со структурой вершины (например, если вершины хранятся непрерывно в памяти, e.g. `std::vector<vertex>`)
- Если часть атрибутов постоянны, а другую часть нужно иногда обновлять – разделите эти части по двум разным буферам
- Если часть атрибутов не нужны для некоторых алгоритмов (например, для теней) – выделите их в отдельный буфер
- Некоторые форматы 3D сцен (напр. **glTF**) хранят вершины в бинарном формате, удобном для загрузки на GPU: поддерживают несколько буферов, разный *stride*, и разные форматы атрибутов

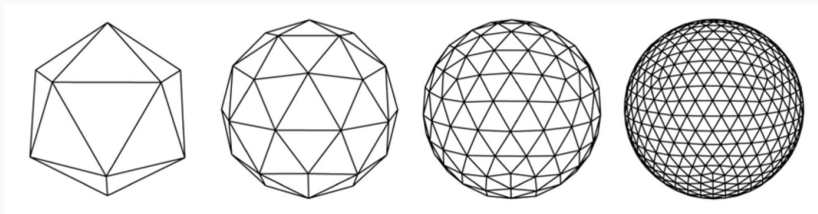
- Для рендеринга нужно выставить буфер с вершинами через `render_pass.set_vertex_buffer(slot, buffer)`

- Для рендеринга нужно выставить буфер с вершинами через `render_pass.set_vertex_buffer(slot, buffer)`
- `slot` – номер *buffer layout*'а в массиве `renderPipelineDescriptor.vertex`

Аттрибуты вершин: пример полностью

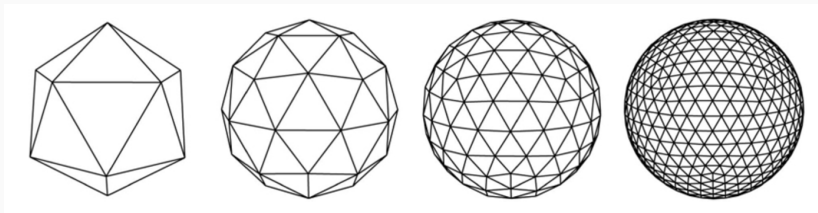
```
// Инициализация:  
pipeline = createPipeline()  
vertexBuffer = loadBuffer(vertices, Usage::Vertex)  
  
// Рендеринг:  
renderPass.setPipeline(pipeline)  
renderPass.setVertexBuffer(vertexBuffer)  
renderPass.draw(vertices.size())
```

Индексированный рендеринг



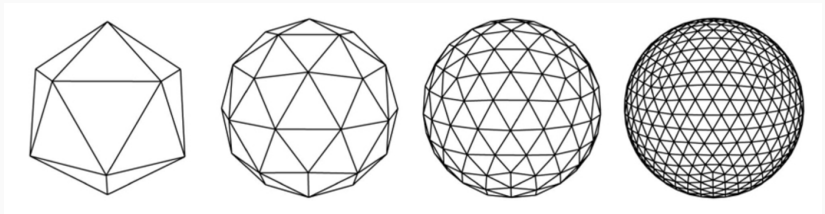
- В моделях одна вершина часто входит в несколько треугольников (в среднем, 6)

Индексированный рендеринг



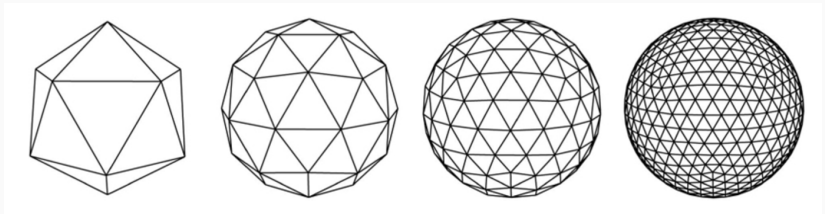
- В моделях одна вершина часто входит в несколько треугольников (в среднем, 6)
- Если использовать примитив `TriangleList`, придётся копировать каждую вершину несколько раз

Индексированный рендеринг



- В моделях одна вершина часто входит в несколько треугольников (в среднем, 6)
- Если использовать примитив `TriangleList`, придётся копировать каждую вершину несколько раз
- `TriangleStrip` только частично решают проблему

Индексированный рендеринг



- В моделях одна вершина часто входит в несколько треугольников (в среднем, 6)
- Если использовать примитив `TriangleList`, придётся копировать каждую вершину несколько раз
- `TriangleStrip` только частично решают проблему
- $\implies$ Индексированный рендеринг

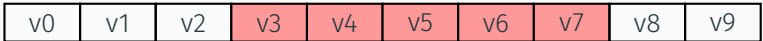
Индексированный рендеринг

- `renderPass.draw(first=3, count=5):`

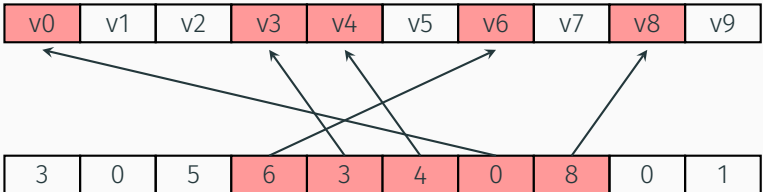
v0	v1	v2	v3	v4	v5	v6	v7	v8	v9
----	----	----	----	----	----	----	----	----	----

Индексированный рендеринг

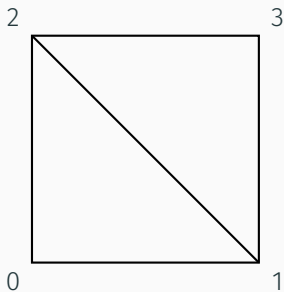
- `renderPass.draw(first=3, count=5):`



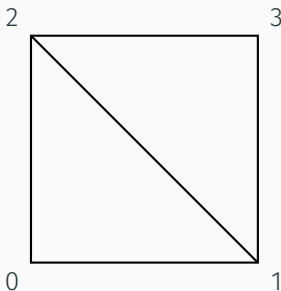
- `renderPass.draw_indexed(first=3, count=5):`



Индексированный рендеринг: квадрат

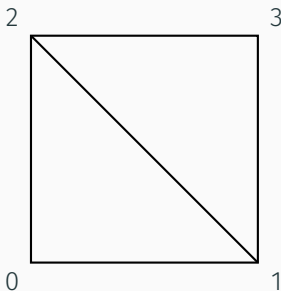


Индексированный рендеринг: квадрат



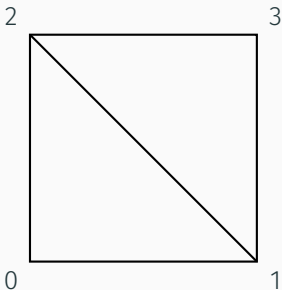
- Без индексов, `TriangleList`:
 - Вершины: `v0`, `v1`, `v2`, `v2`, `v1`, `v3`

Индексированный рендеринг: квадрат



- Без индексов, `TriangleList`:
 - Вершины: `v0`, `v1`, `v2`, `v2`, `v1`, `v3`
- Без индексов, `TriangleStrip`:
 - Вершины: `v0`, `v1`, `v2`, `v3`

Индексированный рендеринг: квадрат



- Без индексов, `TriangleList`:
 - Вершины: `v0`, `v1`, `v2`, `v2`, `v1`, `v3`
- Без индексов, `TriangleStrip`:
 - Вершины: `v0`, `v1`, `v2`, `v3`
- С индексами, `TriangleList`:
 - Вершины: `v0`, `v1`, `v2`, `v3`
 - Индексы: `0`, `1`, `2`, `2`, `1`, `3`

- Индексы хранятся в буфере с `usage = Index`

- Индексы хранятся в буфере с `usage = Index`
- `render_pass.set_index_buffer(buffer, format)` устанавливает текущий буфер с индексами

- Индексы хранятся в буфере с `usage = Index`
- `render_pass.set_index_buffer(buffer, format)` устанавливает текущий буфер с индексами
- Форматов только два: `Uint16` и `Uint32`

Индексированный рендеринг

- Индексы хранятся в буфере с `usage = Index`
- `render_pass.set_index_buffer(buffer, format)` устанавливает текущий буфер с индексами
- Форматов только два: `Uint16` и `Uint32`
- `render_pass.draw_indexed()` вызывает индексированный рендеринг

Индексированный рендеринг: пример

```
// Инициализация:  
pipeline = createPipeline()  
vertexBuffer = loadBuffer(vertices, Usage::Vertex)  
indexBuffer = loadBuffer(indices, Usage::Index)  
  
// Рендеринг:  
renderPass.setPipeline(pipeline)  
renderPass.setVertexBuffer(vertexBuffer)  
renderPass.setIndexBuffer(indexBuffer)  
renderPass.drawIndexed(indices.size())
```

- При использовании `LineStrip` или `TriangleList` все входные вершины образуют один line strip/triangle strip

Primitive restart

- При использовании `LineStrip` или `TriangleList` все входные вершины образуют один line strip/triangle strip
- Что, если мы хотим одну последовательность вершин разбить на несколько примитивов?

Primitive restart

- При использовании `LineStrip` или `TriangleList` все входные вершины образуют один line strip/triangle strip
- Что, если мы хотим одну последовательность вершин разбить на несколько примитивов?
- Можно сделать несколько вызовов `render_pass.draw(...)`, но это расточительно

Primitive restart

- При использовании `LineStrip` или `TriangleList` все входные вершины образуют один line strip/triangle strip
- Что, если мы хотим одну последовательность вершин разбить на несколько примитивов?
- Можно сделать несколько вызовов `render_pass.draw(...)`, но это расточительно
- Максимально возможное для этого типа (`uint16/uint32`) значение индекса означает что со следующего индекса нужно начать новый примитив:
`[0, 1, 2, 3, 65535, 4, 5, 6, 7]` эквивалентно
`[0, 1, 2, 3], [4, 5, 6, 7]`

- Для каждого *типа объекта* (одинаковый набор атрибутов, шейдеры) – свой pipeline

- Для каждого *типа* объекта (одинаковый набор атрибутов, шейдеры) – свой pipeline
- Для каждого *отдельного* объекта – свой набор буферов (напр. vertex + index)

Рендеринг нескольких объектов

- Для каждого *типа объекта* (одинаковый набор атрибутов, шейдеры) – свой pipeline
- Для каждого *отдельного объекта* – свой набор буферов (напр. vertex + index)
- Можно иметь общий буфер на все вершины и индексы всех объектов, если их не нужно обновлять, и делать `render_pass.set_vertex_buffer(0, buffer, offset=...)`

- У настройки *buffer layout*'а есть ещё один параметр – *step mode*

- У настройки *buffer layout*'а есть ещё один параметр – *step mode*
- Значение `stepMode = vertex` означает, что атрибуты будут браться по *номеру вершины*

- У настройки *buffer layout*'а есть ещё один параметр – *step mode*
- Значение `stepMode = vertex` означает, что атрибуты будут браться по *номеру вершины*
- Значение `stepMode = instance` означает, что атрибуты будут браться по *номеру инстанса*

- *Instancing*, он же *instanced rendering* позволяет одним вызовом нарисовать много копий объекта

- *Instancing*, он же *instanced rendering* позволяет одним вызовом нарисовать много копий объекта
- Типичная разбивка атрибутов:
 - `stepMode = vertex` – атрибуты вершин 3D-модели: позиции, нормали, цвета
 - `stepMode = instance` – атрибуты каждой отдельной копии: позиция, поворот, дополнительный цвет

- *Instancing*, он же *instanced rendering* позволяет одним вызовом нарисовать много копий объекта
- Типичная разбивка атрибутов:
 - `stepMode = vertex` – атрибуты вершин 3D-модели: позиции, нормали, цвета
 - `stepMode = instance` – атрибуты каждой отдельной копии: позиция, поворот, дополнительный цвет
- Как правило, `vertex` и `instance` атрибуты лежат в разных буферах

Instancing

- *Instancing*, он же *instanced rendering* позволяет одним вызовом нарисовать много копий объекта
- Типичная разбивка атрибутов:
 - `stepMode = vertex` – атрибуты вершин 3D-модели: позиции, нормали, цвета
 - `stepMode = instance` – атрибуты каждой отдельной копии: позиция, поворот, дополнительный цвет
- Как правило, `vertex` и `instance` атрибуты лежат в разных буферах
- Вызов `render_pass.draw(vertex_count=N, instance_count=M)` обработает $N \cdot M$ вершин: M копий модели из N вершин

- Раньше instancing имел значительные накладные расходы, и использовался только для большого количества копий (сотен или тысяч)

- Раньше instancing имел значительные накладные расходы, и использовался только для большого количества копий (сотен или тысяч)
- В современных API instancing – основная функциональность, все функции рисования (`render_pass.draw()`) по умолчанию работают с instancing'ом